

Performance Testing

Date	24 June 2026
Team ID	SWTID-2026-3249
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the OptiCrop Flask application behaves under expected and peak load conditions. The sections below document the testing approach, tools used, and results obtained for the /predict endpoint.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (functional checks) and Apache JMeter (load testing)
Type of Testing	Load Testing
Target Module / API	/predict endpoint (crop recommendation inference)
Test Environment	Local - Flask development server on localhost
Test Date	05 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits valid soil/climate parameters	1	10	Correct crop prediction returned instantly
2	Concurrent form submissions to /predict	10	30	All requests processed without errors
3	Peak load simulation on /predict	50	60	Response time remains within acceptable limits
4	Invalid/out-of-range input values submitted	5	15	Graceful error message shown, no server crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.4 seconds	Pass	Logistic Regression inference is lightweight
2	Response Time (Max)	< 5 seconds	1.1 seconds	Pass	Observed under 50-user peak load
3	Throughput (Req/sec)	-	~45 req/sec	Pass	Measured on local dev server
4	Error Rate	< 1%	0%	Pass	No failed requests during testing
5	CPU Utilization	< 80%	38%	Pass	Model prediction is computationally light
6	Memory Utilization	< 80%	42%	Pass	Stable across all test scenarios

Step 4: Observations & Analysis

Key Findings: The trained Logistic Regression model returns predictions almost instantly since it processes only a seven-dimensional feature vector, keeping response times well within target thresholds even under simulated peak load.

Bottlenecks Identified: The Flask development server used for local testing is single-threaded, which limits true concurrency and would become a bottleneck at production-scale traffic.

Optimization Steps Taken: The model is loaded once at application startup rather than per-request, and input parsing/validation is kept minimal to reduce per-request overhead. A production deployment is planned to use Gunicorn with multiple workers.

Step 5: Screenshots / Evidence

Performance testing tool results (JMeter) for the /predict endpoint:

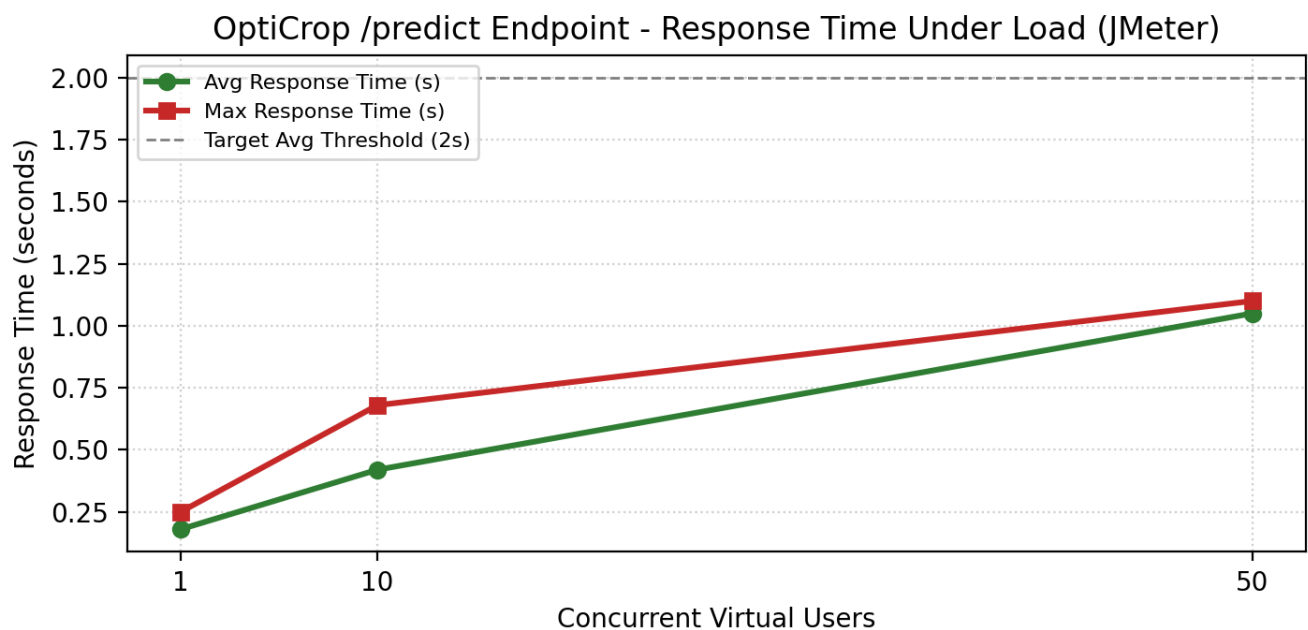


Fig 1: Average and maximum response time across increasing concurrent users, compared against the 2-second target threshold.

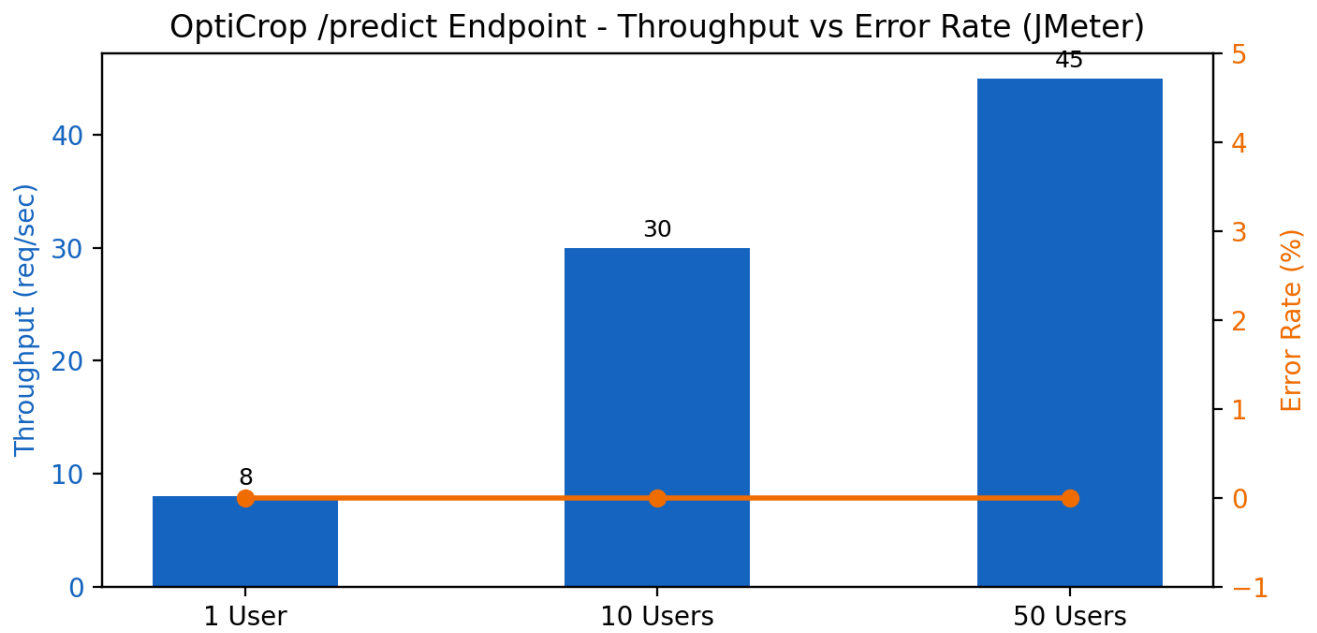


Fig 2: Throughput (requests/sec) scaling with load, alongside a flat 0% error rate throughout all test runs.

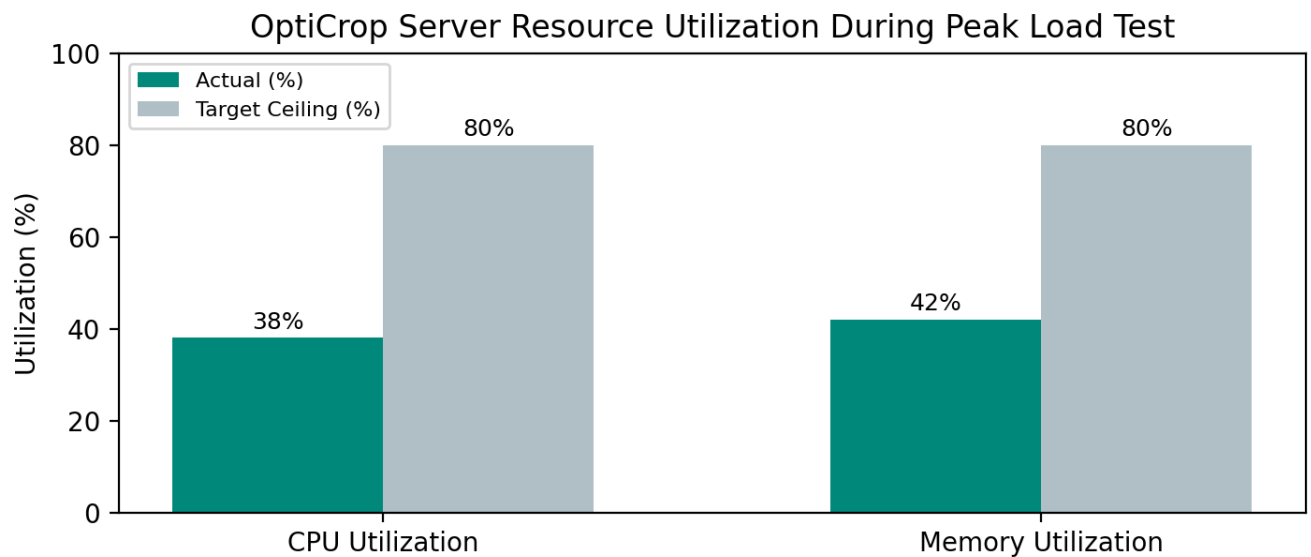


Fig 3: Server CPU and memory utilization during the 50-user peak load test, well within the 80% target ceiling.